

---

MSC DISSERTATION · TECHNICAL SYSTEM REPORT

# The Adaptive Chess Puzzle Advisor

*A Bayesian bandit-driven system for diagnosing tactical weakness  
from real game history and adaptively targeting remedial practice*

---

|                    |                                                  |
|--------------------|--------------------------------------------------|
| <b>Author</b>      | Aleksandar Radevski                              |
| <b>Programme</b>   | MSc Artificial Intelligence & Data Science       |
| <b>Institution</b> | City College, University of York                 |
| <b>Period</b>      | July – October 2026                              |
| <b>Document</b>    | Technical system report, algorithms & evaluation |
| <b>Date</b>        | 8 September 2026                                 |

---

Every constant, metric and behavioural claim in this report was read directly from the current source tree and is cross-referenced to the file and function that implements it.

# Abstract

Chess improvement literature and expertise research converge on a single prescription: practice is effective in proportion to how precisely it targets a learner's actual deficiencies<sup>[19]</sup>. Mainstream chess platforms, however, serve tactical puzzles drawn from difficulty-matched pools without any model of *which* tactical motifs a given player actually fails at. This report documents the **Adaptive Chess Puzzle Advisor**, a working end-to-end system that closes that gap.

The system ingests a player's Chess.com game history, analyses every move with Stockfish to produce a per-category weakness profile across a 23-motif tactical taxonomy, and then drives an adaptive practice session using a **Thompson Sampling multi-armed bandit**<sup>[1][2]</sup> in which each tactical category is an independent Beta–Bernoulli arm. Practice material comes from two sources: a filtered and re-categorised 6-million-row Lichess puzzle corpus<sup>[14]</sup>, and puzzles mined from the player's own losses through a four-gate Stockfish verification pipeline that rejects dual-solution and trivially-forced positions.

Two further components exist as validated research artefacts rather than default production paths: a from-scratch **Glicko-2**<sup>[5][6]</sup> difficulty fitter that re-estimates puzzle difficulty from the system's own solve logs, validated against Glickman's published worked example, and a **RandomForest**<sup>[8]</sup> weakness classifier validated by held-out-player cross-validation on simulated cohorts. Both are gated on real-player data volume, not on unfinished implementation — a distinction this report states explicitly rather than eliding.

The report specifies each algorithm in full mathematical detail, documents the three pipelines and how they interconnect, gives an honest component-by-component account of which parts are genuinely learned from data and which are symbolic, and grounds every design decision in the primary literature.

**23**

TACTICAL CATEGORIES  
(BANDIT ARMS)

**6M+**

PUZZLES FILTERED  
& RE-CATEGORISED

**128**

PASSING TESTS  
ACROSS 8 FILES

# Contents

## 1. Introduction

- 1.1 Problem statement
- 1.2 Why adaptivity is the right response
- 1.3 Research objectives
- 1.4 Contributions

## 2. Background and Related Work

- 2.1 Rating systems: Elo, Glicko, Glicko-2
- 2.2 Multi-armed bandits and Thompson Sampling
- 2.3 Chess engines: alpha-beta search and NNUE evaluation
- 2.4 Adaptive learning and knowledge tracing
- 2.5 Positioning of this work

## 3. System Architecture

- 3.1 Component overview
- 3.2 The three pipelines and their coupling
- 3.3 Persistence model
- 3.4 Technology stack

## 4. Data Layer

- 4.1 The Lichess Open Puzzle Database
- 4.2 ETL: streaming construction of the served pool
- 4.3 The 23-category weakness taxonomy
- 4.4 The motif-priority defect and its correction
- 4.5 Chess.com ingestion

## 5. Pipeline 1 — Player Weakness Profiling

- 5.1 PGN parsing and phase classification
- 5.2 Per-move Stockfish analysis
- 5.3 The rule-based weakness scorer
- 5.4 The trained RandomForest classifier
- 5.5 The engine-free fallback and path selection

## 6. Pipeline 2 — Puzzle Mining from Real Games

- 6.1 Rationale: verified extraction, not generation
- 6.2 The four verification gates
- 6.3 Tactic classification and rating estimation

## 7. Pipeline 3 — Adaptive Recommendation

- 7.1 Formalisation as a Bernoulli bandit
- 7.2 Beta–Bernoulli conjugacy
- 7.3 The selection rule and its deliberate inversion
- 7.4 Prior seeding from the weakness profile
- 7.5 The serving cascade

## 8. Difficulty Modelling with Glicko-2

- 8.1 Why a three-parameter rating system
- 8.2 The update algorithm in full
- 8.3 Adaptation to puzzle rating
- 8.4 Validation against the published reference

## 9. Evaluation

- 9.1 A two-level evaluation strategy
- 9.2 Puzzle quality: the five-dimension evaluator
- 9.3 Recommender validation methodology
- 9.4 Classifier cross-validation results
- 9.5 Test suite and reference validation

## 10. What Is Genuinely AI-Driven

- 10.1 Component-by-component breakdown
- 10.2 The defensible claim

## 11. Limitations and Threats to Validity

- 11.1 Defects found and fixed
- 11.2 Open limitations

## 12. Future Work

## 13. Conclusion

## 14. References

## 15. Appendices

- 15.1 Literature-to-component mapping
- 15.2 Complete constants reference
- 15.3 Repository map
- 15.4 Reproduction guide

# 1 Introduction

## 1.1 Problem statement

A club-level chess player who wants to improve faces an information problem before they face a training problem. They know they lose games; they do not reliably know *why*. Self-diagnosis is unreliable precisely because the errors that cost games are, by definition, the ones the player did not see. The standard remedy — solving tactical puzzles — is offered by every major platform, but the selection policy is almost universally *difficulty-matched* rather than *weakness-matched*: puzzles are drawn from a pool filtered by rating, with no model of which tactical motifs this particular player systematically misses.

The consequence is a misallocation of practice time. A player who is competent at forks and pins but loses repeatedly to back-rank mates and rook endgames will, under a rating-matched policy, receive forks and pins in proportion to their frequency in the corpus — which is to say, mostly. Their practice is comfortable, measurable, and largely redundant.

## 1.2 Why adaptivity is the right response

Ericsson, Krampe and Tesch-Römer's account of deliberate practice<sup>[19]</sup> identifies the defining property of practice that produces expertise: it is targeted at tasks just beyond current competence, with immediate feedback, and it is specifically *not* the repetition of already-mastered skills. Vygotsky's zone of proximal development<sup>[18]</sup> supplies the complementary constraint from the other direction — material far above current ability produces failure without learning. An effective puzzle recommender must therefore solve two coupled problems simultaneously: **what** motif to practise (the weakness-targeting problem) and **how hard** the instance should be (the difficulty-matching problem).

Intelligent tutoring systems have addressed the first problem since Corbett and Anderson's knowledge tracing framework<sup>[17]</sup>, which maintains a per-skill probability that a learner has mastered each component skill and updates it from observed successes and failures. This system adopts the same fundamental posture — a per-skill belief updated online from evidence — but implements it as a Bayesian bandit, which additionally answers the *selection* question that knowledge tracing alone leaves open: given current beliefs, which skill should be exercised next, and how should the system trade off acting on what it believes against testing what it is unsure of?

### The central design decision

Weakness is never directly observable. It is inferred from noisy binary evidence (solved / failed), and every observation costs the learner time. This is precisely the structure of a multi-armed bandit problem, which is why the recommendation core of this system is a Thompson Sampling bandit<sup>[1]</sup> rather than a ranking heuristic over a static profile.

## 1.3 Research objectives

| Adaptive Chess Puzzle Advisor — Technical System Report |                                                                         | A. Radevski · MSc AI & Data Science |                                                                                                                                                          |
|---------------------------------------------------------|-------------------------------------------------------------------------|-------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| #                                                       | Objective                                                               | Status at time of writing           |                                                                                                                                                          |
| 1                                                       | Classify a player's tactical weakness profile from their game history   | <b>Met</b>                          | Rule-based Stockfish-driven scorer live by default; trained RandomForest alternative validated on simulated cohorts and reachable via opt-in flag (§5.4) |
| 2                                                       | Produce legal, unique, non-trivial puzzles targeted at those weaknesses | <b>Met, reframed</b>                | Engine-verified mining from the player's own games rather than generative synthesis — a deliberate, documented scope decision (§6.1)                     |
| 3                                                       | Match puzzle difficulty to player ability                               | <b>Partially met</b>                | Rating-band targeting live; Glicko-2 fitting from the system's own solve logs built and validated but not yet in the serving path (§8)                   |
| 4                                                       | Evaluate the adaptive system against a non-adaptive baseline            | <b>Partially met</b>                | Five-dimension puzzle quality evaluator live; bandit simulation methodology specified with pre-registered criteria (§9.3)                                |

## 1.4 Contributions

1. **An end-to-end working system** connecting public game-history ingestion, engine-based error analysis, weakness profiling, puzzle mining, and adaptive serving — deployed as a Flask application with a browser front end.
2. **A Thompson Sampling formulation of tactical weakness targeting**, including a deliberate inversion of the standard reward-maximising selection rule (§7.3) and a principled scheme for seeding arm priors from engine-derived error statistics (§7.4).
3. **A four-gate engine verification pipeline** for mining sound puzzles from amateur games, which eliminated the ~40% false-positive rate of an earlier pattern-matching approach (§6).
4. **A from-scratch Glicko-2 implementation** adapted to treat puzzles as opponents, validated to published-reference tolerance against Glickman's own worked example (§8.4).
5. **A documented correction to a taxonomy defect** that had mislabelled approximately 198,000 puzzles and would have systematically corrupted the bandit's weakness model (§4.4).

### A note on the evidential standard of this document

Where the implementation diverges from what earlier project documentation claimed, this report states the divergence rather than smoothing it over. Section 11 documents defects found by re-reading the live source, including two that made the system's primary analysis path unreachable in practice. This is deliberate: a technical report whose claims cannot be falsified against the source tree is not evidence of anything.

## 2 Background and Related Work

### 2.1 Rating systems: Elo, Glicko, Glicko-2

Elo's rating system<sup>[7]</sup> models the outcome of a paired comparison as a function of the difference between two scalar strength parameters, updating each after every result by a fixed-gain rule. Its limitation is structural: a single number cannot distinguish a rating established over a thousand games from one assigned after two. Both are simply numbers, and both move by the same increment under the same result.

Glickman's Glicko system<sup>[5]</sup> addresses this by modelling strength as a distribution rather than a point, adding a *rating deviation* (RD) that quantifies the system's uncertainty. Ratings with high RD move quickly under new evidence; well-established ratings move slowly. Glicko-2<sup>[6]</sup> extends this with a third parameter, *volatility*, capturing how erratically the underlying true strength itself appears to change — separating "this result was ordinary variance" from "this competitor's real strength is genuinely shifting."

This distinction matters directly for puzzle rating. A newly mined puzzle has been attempted by nobody; its difficulty is a guess. A Lichess puzzle with 40,000 recorded attempts has a difficulty known to within a few points. Under Elo these are indistinguishable; under Glicko-2 the difference is explicit and actionable. Section 8 develops the algorithm and its adaptation to this system.

### 2.2 Multi-armed bandits and Thompson Sampling

The multi-armed bandit problem formalises sequential decision-making under uncertainty: an agent repeatedly chooses among actions with unknown reward distributions, and must balance *exploitation* of the action currently believed best against *exploration* that reduces uncertainty about the others<sup>[4]</sup>.

Thompson's 1933 paper<sup>[1]</sup> — predating the formal bandit literature by decades — proposed what is now the canonical Bayesian solution: maintain a posterior distribution over each action's reward probability, draw one sample from each posterior, and act greedily with respect to the sampled values. The elegance of this rule is that exploration is not a bolt-on. An action whose posterior is wide is sampled across a wide range and will occasionally win the comparison on its own merits; as evidence accumulates the posterior narrows and the same rule becomes increasingly exploitative, with no schedule, no temperature, and no tunable exploration constant.

The principal alternative family, Upper Confidence Bound methods, achieves logarithmic regret through deterministic optimism<sup>[3]</sup>. Chapelle and Li's empirical study<sup>[20]</sup> found Thompson Sampling competitive with or superior to UCB across realistic problem settings, and Russo *et al.*'s tutorial<sup>[2]</sup> supplies the modern regret-bound treatment together with practical guidance. Two properties made it the right choice here: rewards are exactly Bernoulli (a puzzle is solved or not), which makes the Beta–Bernoulli conjugate update exact and closed-form; and there is no exploration parameter requiring calibration against user data this project does not yet have at scale.

## 2.3 Chess engines: alpha-beta search and NNUE evaluation

All ground truth in this system — what the best move was, how much a played move cost, whether a candidate puzzle has a unique solution — comes from Stockfish<sup>[12]</sup>, an alpha-beta searcher whose evaluation function since version 12 is an NNUE network: an efficiently-updatable shallow neural evaluator originally developed by Nasu for computer shogi<sup>[11]</sup> and subsequently adapted to chess. The relevant property for this work is incremental updatability, which makes NNUE evaluation cheap enough to remain viable inside a high-throughput alpha-beta search — a practical requirement given that this system analyses tens of thousands of positions per user.

It is worth stating what this system does *not* do. AlphaZero<sup>[10]</sup> demonstrated that a policy-value network trained by self-play with Monte Carlo tree search can reach superhuman chess strength from rules alone. That is a fundamentally different problem from the one addressed here. This project does not train a chess-playing agent; it treats a mature engine as an oracle and applies machine learning to the *pedagogical* layer above it — which player needs which practice, and when.

## 2.4 Adaptive learning and knowledge tracing

Corbett and Anderson's knowledge tracing<sup>[17]</sup> established the template for modelling learner mastery in intelligent tutoring systems: a latent per-skill mastery probability, updated by Bayesian inference from observed correct and incorrect responses, with explicit parameters for guessing and slipping. The Beta-Bernoulli arms in this system are a direct structural analogue — a per-skill posterior over success probability, updated from binary evidence.

The substantive difference is decision-theoretic. Knowledge tracing estimates mastery; it does not by itself specify a selection policy. By formulating selection as a bandit, this system obtains an exploration-aware policy for free, which matters when the number of skills (23) is large relative to the number of observations available in a single practice session (tens).

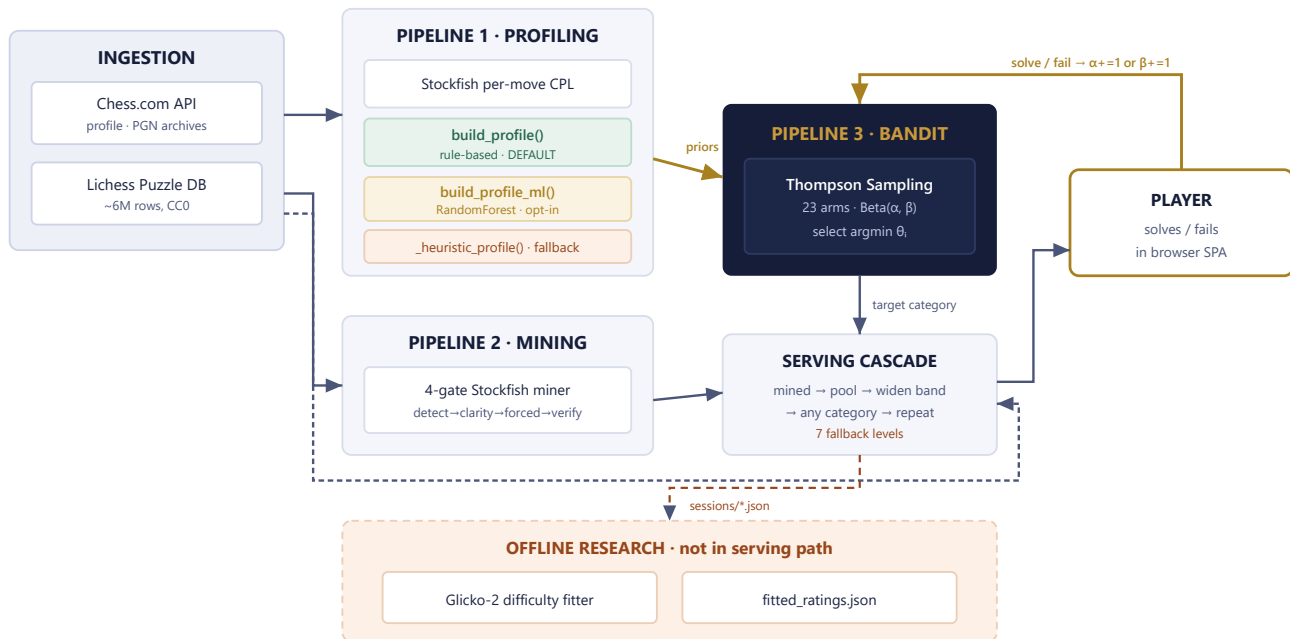
## 2.5 Positioning of this work

| Prior art                                       | What it establishes                                           | How this system relates                                                                 |
|-------------------------------------------------|---------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| Elo <sup>[7]</sup> , Glicko-2 <sup>[5][6]</sup> | Uncertainty-aware strength estimation from paired comparisons | Reimplemented from scratch and adapted to treat a <i>puzzle</i> as the opponent (§8.3)  |
| Thompson Sampling <sup>[1][2]</sup>             | Bayesian exploration/exploitation with no tuning parameter    | Applied with an inverted objective — select the <i>weakest</i> arm, not the best (§7.3) |
| Knowledge tracing <sup>[17]</sup>               | Per-skill Bayesian mastery estimation in tutoring systems     | Same posture, extended with an explicit selection policy over 23 skills                 |
| AlphaZero <sup>[10]</sup>                       | Self-play RL can master chess itself                          | Explicitly not attempted; the engine is an oracle, ML operates on the pedagogical layer |
| Lichess puzzle corpus <sup>[14]</sup>           | 6M engine-verified, community-rated puzzles under CCO         | Used as the base pool, re-categorised under a purpose-built 23-motif taxonomy (§4.3)    |

## 3 System Architecture

### 3.1 Component overview

The system decomposes into three pipelines that meet at a single shared object — the per-user bandit state. Pipeline 1 produces a weakness profile that seeds the bandit's priors. Pipeline 2 mines personalised puzzle material. Pipeline 3 consumes both, serving puzzles and feeding outcomes back into the bandit posterior. Nothing else is stateful.



**Figure 1.** System architecture. Solid arrows are live data flow; dashed arrows indicate the offline research path that consumes session logs but does not yet feed serving.

### 3.2 The three pipelines and their coupling

The pipelines are deliberately loosely coupled. Pipeline 1 runs once per analysis request, in a background thread, and terminates by writing a set of Beta priors. Pipeline 2 runs independently and writes a JSON file of mined puzzles. Pipeline 3 is the only long-lived interactive loop, and it degrades gracefully if either upstream pipeline produced nothing: absent priors, arms start at the uniform `Beta(1,1)`; absent mined puzzles, the cascade falls through to the Lichess pool.

This matters architecturally because Stockfish analysis is expensive — roughly three to five minutes for fifty games at two worker threads. Binding the interactive loop to that latency would make the system unusable. Instead, analysis is asynchronous and its result is a *prior*, which by construction the bandit is designed to override as real evidence arrives (§7.4).

### 3.3 Persistence model

There is no database. All state is JSON on disk, which is a defensible choice at this scale and an explicit limitation at any larger one (§11.2).

| Path                               | Contents                                             | Written by                                  |
|------------------------------------|------------------------------------------------------|---------------------------------------------|
| data/sessions/<user>.json          | Bandit arm states, solve history, streaks            | Every <code>/api/session/result</code> call |
| data/user_puzzles/<user>.json      | Mined puzzles, deduplicated by <code>PuzzleId</code> | Pipeline 2; additive, never destructive     |
| data/cache/chess_com/              | Cached API responses                                 | Ingestion layer, rate-limited               |
| data/processed/*.parquet           | Cleaned, enriched, split puzzle pool                 | One-shot ETL script (§4.2)                  |
| data/processed/fitted_ratings.json | Glicko-2 fitted puzzle/player ratings                | Offline CLI only — not read at serve time   |

### 3.4 Technology stack

| Layer                 | Technology                                                               | Reference |
|-----------------------|--------------------------------------------------------------------------|-----------|
| Core language         | Python 3.13                                                              | —         |
| Board logic, PGN, UCI | <code>python-chess</code>                                                | [13]      |
| Engine oracle         | Stockfish 18 (bundled binary)                                            | [12][11]  |
| Recommender           | Custom Thompson Sampling over <code>numpy.random.beta</code>             | [1][2]    |
| Difficulty rating     | Custom from-scratch Glicko-2, no external library                        | [5][6]    |
| Weakness classifier   | <code>scikit-learn</code> <code>RandomForestRegressor</code>             | [8][9]    |
| Data processing       | <code>pandas</code> , <code>pyarrow</code> (Parquet row-group streaming) | [16]      |
| Backend / frontend    | Flask + flask-cors; vanilla JS SPA, same origin                          | —         |
| Testing               | <code>pytest</code> — 128 tests, 8 files, fully offline                  | —         |

## 4 Data Layer

### 4.1 The Lichess Open Puzzle Database

The base puzzle corpus is the Lichess Open Database puzzle export<sup>[14]</sup>: approximately 6 million puzzles released under CCo, each carrying a FEN starting position, a solution move sequence in UCI, a Glicko-2 rating with its rating deviation, a popularity score, a play count, and a set of theme tags. The corpus is itself machine-generated — Lichess derives it by scanning hundreds of millions of played games for positions where a decisive tactical opportunity exists, then re-analysing those positions with Stockfish NNUE at high node counts.

This provenance is significant for the present work in two ways. It means the puzzles are already engine-verified, so the quality problem for pool puzzles reduces to *filtering* rather than *verification*. And it means the supplied ratings are Glicko-2 ratings fitted from a very large population of Lichess solvers — statistically strong, but fitted on a different population than this system's users, which is precisely the motivation for the difficulty refitting work in §8.

### 4.2 ETL: streaming construction of the served pool

The raw CSV is roughly 1.1 GB, which forbids naive whole-file loading. The ETL script streams it in one-million-row chunks, enriching and cleaning each chunk before writing Parquet outputs partitioned three ways — a single full table, one file per category, and one file per difficulty tier — plus a category summary.

The cleaning stage drops rows with missing FEN, moves or rating; deduplicates on `PuzzleId`; drops puzzles whose rating deviation exceeds 150 (poorly calibrated); drops puzzles never actually played; and requires a non-empty solution. At server start the application performs a second pass, reading Parquet *one row-group at a time* so the full six million rows are never materialised, applying a live quality gate and sampling proportionally down to a bounded pool. A `PUZZLE_BY_CAT` index is then built for constant-time category lookup during serving.

### 4.3 The 23-category weakness taxonomy

Lichess emits roughly fifty raw theme tags mixing motif, phase, length and outcome. These are mapped onto a purpose-built taxonomy of 23 categories, which becomes the shared vocabulary of the entire system: the arms of the bandit, the keys of the weakness profile, the labels of mined puzzles, and the output classes of the trained classifier all range over exactly this set.

| Count     | Category       | Count   | Category          | Count   | Category       |
|-----------|----------------|---------|-------------------|---------|----------------|
| 2,942,587 | Endgame        | 330,422 | Discovered Attack | 113,304 | Queen Endgame  |
| 1,849,329 | Mating Pattern | 293,854 | Deflection        | 91,293  | Interference   |
| 766,027   | King Safety    | 278,677 | Hanging Piece     | 81,399  | Bishop Endgame |
| 758,947   | Fork           | 247,767 | Quiet Move        | 78,289  | Clearance      |
| 447,248   | Sacrifice      | 219,843 | Pawn Endgame      | 62,068  | Zugzwang       |
| 362,109   | Rook Endgame   | 216,012 | Attraction        | 49,229  | Knight Endgame |
| 355,611   | Pin            | 142,800 | Promotion         | 21,100  | X-Ray Attack   |
| 130,482   | Skewer         | 8,375   | En Passant        |         |                |

The distribution is severely imbalanced — Endgame and Mating Pattern together account for more than half the corpus, while En Passant and X-Ray Attack are three orders of magnitude rarer. This is a real constraint on the recommender: for the rarest categories, the pool may contain too few puzzles in a given rating band to serve a full session, which is exactly what the serving cascade's band-widening fallbacks exist to handle (§7.5).

## 4.4 The motif-priority defect and its correction

### A defect that would have silently corrupted the weakness model

Lichess emits theme tags in alphabetical order. The original enrichment logic resolved a puzzle's primary category by taking the first recognised tag — which meant the alphabetically-early *phase* tag `endgame` outranked genuine motif tags. A puzzle tagged `crushing endgame fork short` was filed as **Endgame** rather than **Fork**.

Approximately **198,000 of 305,000** puzzles then carrying the Endgame label — around 45% of that category — were misclassified. The consequence is not merely cosmetic: a player served these puzzles would have solve/fail evidence attributed to the wrong bandit arm, systematically corrupting the weakness posterior that the entire system exists to estimate.

The correction introduces an explicit specificity ordering, `MOTIF_PRIORITY`, resolving ties in favour of the most informative label: forced mate ranks above a concrete tactical motif, which ranks above a generic attacking theme, which ranks above a specific endgame type, which ranks above bare "endgame", with "General" as terminal fallback. Because Parquet files built before the fix would otherwise retain bad labels, the resolver is re-run over the pool in memory at application load time, so a stale artefact is corrected rather than trusted.

## 4.5 Chess.com ingestion

Player data comes from the Chess.com Published-Data API<sup>[15]</sup>, which is fully public and requires no authentication. The client fetches the player profile and walks monthly game archives newest-first, stopping once the requested game count is reached, with a 0.5-second inter-request delay in line with the platform's

rate-limit guidance. Every response is cached to disk, so repeated analyses of the same player never re-hit the network — which matters both for courtesy to the upstream service and for the reproducibility of any evaluation run.

## 5 Pipeline 1 — Player Weakness Profiling

*Input: a Chess.com username. Output: a map from each of 23 categories to a weakness score in  $[0, 1]$ , where 0 is strong and 1 is weak — which is then converted into Beta priors for the bandit.*

### 5.1 PGN parsing and phase classification

Parsing extracts player colour, result, ratings, and opening identity (ECO code plus a family name resolved through a lookup table). Game phase is classified purely by piece count: 28 or more pieces is opening, 14 to 27 is middlegame, fewer than 14 is endgame. The important property is that this single function is shared by both the analyser and the puzzle miner, so "phase" denotes exactly the same thing everywhere in the system — a small consistency guarantee that avoids a class of subtle attribution bugs.

### 5.2 Per-move Stockfish analysis

For every move the player made — skipping the first eight half-moves as opening theory rather than tactical decision-making — the analyser evaluates the position before the move, records the engine's preferred continuation, plays the move actually chosen, evaluates again, and takes the difference as *centipawn loss*:

$$cp\_loss = score_{before} - score_{after}$$

Errors are then graded against fixed thresholds and recorded with their phase and, critically, their tactical category — the missed best move is classified through the *same* routine the puzzle miner uses, so "which motif did I fail to see" is answered identically whether the position came from game analysis or from a mined puzzle.

| Constant      | Value  | Meaning                                           |
|---------------|--------|---------------------------------------------------|
| ANALYSIS_TIME | 0.05 s | Engine time per position ( $\approx$ depth 10–12) |
| INACCURACY_CP | 50 cp  | Threshold for recording a <code>MoveError</code>  |
| MISTAKE_CP    | 100 cp | Severity boundary: mistake                        |
| BLUNDER_CP    | 200 cp | Severity boundary: blunder                        |
| SKIP_PLIES    | 8      | Opening half-moves excluded from analysis         |

Analysis is parallelised across worker threads with one Stockfish subprocess per thread — no shared engine state, therefore no locking. Fifty games complete in roughly three to five minutes at two workers.

### 5.3 The rule-based weakness scorer

The default scorer aggregates per-move errors into a profile and derives weakness scores through hand-tuned rules. All 23 categories begin at a neutral 0.5 and are adjusted upward by additive boosts, clamped at 0.90–0.92 depending on the rule that fired.

| Signal                      | Rule                              | Categories affected                                                                     |
|-----------------------------|-----------------------------------|-----------------------------------------------------------------------------------------|
| Middlegame error rate > 0.5 | <code>+min(0.25, rate×0.3)</code> | Fork, Pin, Skewer, Discovered Attack, Sacrifice, Deflection, Hanging Piece, King Safety |
| Endgame error rate > 0.25   | <code>+min(0.30, rate×0.4)</code> | Endgame and all five specific endgame types                                             |
| Blunders per game ≥ 2       | <code>+0.25</code>                | Hanging Piece, Fork, Pin                                                                |
| Blunders per game ≥ 1       | <code>+0.15</code>                | Hanging Piece, Fork                                                                     |
| Accuracy estimate < 70      | <code>+0.10</code>                | Fork, Pin, Skewer, Discovered Attack                                                    |

This is deliberately coarse and deliberately conservative. It has no notion of a ground-truth label against which an accuracy could ever be reported, which is precisely the methodological objection that motivated building a trained alternative.

## 5.4 The trained RandomForest classifier

The learned alternative is a `RandomForestRegressor` [8][9] predicting all 23 category weaknesses simultaneously from a 30-dimensional feature vector: seven coarse summary statistics (middlegame and endgame error rates, blunders, mistakes and inaccuracies per game, accuracy estimate, log games analysed) plus 23 per-category observed error rates. It is implemented as a strict drop-in — same signature, same return type, overriding only the weakness scores, with every other profile field unit-tested to be identical.

### Why the training data is simulated, and why that is disclosed rather than obscured

Only four real players currently have mined-puzzle history, at 9–33 puzzles each. A held-out-player split at  $n = 4$  cannot yield a meaningful accuracy, error bar or confidence interval. This was verified before any model code was written. Training therefore uses synthetic players generated with a known latent weakness vector, whose observed features are drawn through a Dirichlet-multinomial process — concentration boosted for genuinely weak categories, but sparse and noisy for players with few recorded errors, mirroring the real data's sparsity.

The consequence is stated plainly: the cross-validation results in §9.4 validate the *methodology* — the feature design, the held-out-player protocol, the simulation linking coarse and per-category signal. They are not a claim about accuracy on real players.

## 5.5 The engine-free fallback and path selection

A third path exists for when Stockfish cannot be located or the engine call fails. It inspects only win/loss and game-length patterns from PGN headers, with no board analysis at all, and assigns a single shared score to every tactical category and another to every endgame category:

```
tactical_w = clamp(0.75 - win_rate×0.50 + short_loss_rate×0.20, 0.30, 0.85)
endgame_w  = clamp(0.60 - win_rate×0.30 + long_loss_rate×0.30, 0.25, 0.80)
```

There is no per-category differentiation whatsoever in this path — it is a last resort, and §11.1 documents a defect that caused it to run almost universally in practice for a period.

| Path                              | Nature                                     | Status in the live application                                             |
|-----------------------------------|--------------------------------------------|----------------------------------------------------------------------------|
| <code>build_profile()</code>      | Rule-based, per-category, engine-driven    | <b>Default</b> on every successful analysis                                |
| <code>build_profile_ml()</code>   | Trained RandomForest, per-category         | <b>Opt-in</b> via <code>WEAKNESS_MODEL=ml</code> ; falls back on any error |
| <code>_heuristic_profile()</code> | Win/loss heuristic, two buckets, no engine | <b>Last resort</b> only if the engine is unavailable                       |

The status endpoint reports which of the three actually produced a given profile, so the provenance of any weakness map is inspectable per run rather than inferred.

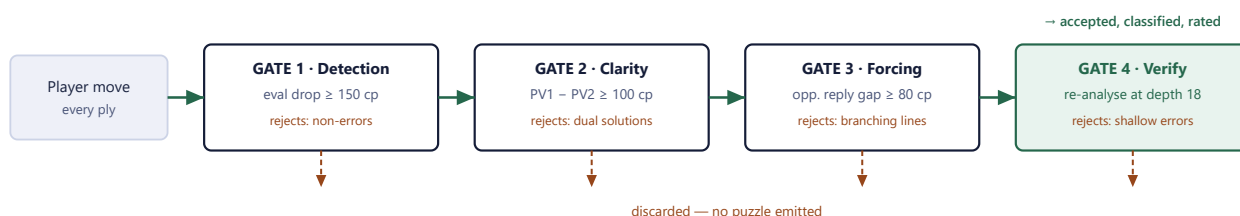
## 6 Pipeline 2 — Puzzle Mining from Real Games

### 6.1 Rationale: verified extraction, not generation

An earlier iteration attempted heuristic puzzle detection using pure board pattern-matching with no engine — fast, but producing roughly **40% false positives**: positions whose nominal "solution" was not in fact the best move. That mode was removed entirely rather than tuned.

The current pipeline extracts puzzles exclusively from positions where the player themselves went wrong, and subjects every candidate to four independent engine gates. This is a deliberate scope decision worth stating precisely, because it differs from a generative framing: the system performs *engine-verified extraction from the player's own history*. "Novel" here means new to this player, not synthesised by a learned generative model. Stating this explicitly is preferable to leaving a generative claim implied and unmet.

### 6.2 The four verification gates



**Figure 2.** The four-gate mining pipeline. Each gate is an independent engine query; a candidate must survive all four to be emitted as a puzzle.

| Constant           | Value  | Role                                                                    |
|--------------------|--------|-------------------------------------------------------------------------|
| PUZZLE_THRESHOLD   | 150 cp | Gate 1 — minimum eval drop to qualify as a puzzle-worthy error          |
| MIN_CLARITY_CP     | 100 cp | Gate 2 — PV1 must beat PV2 by this margin (uniqueness)                  |
| FORCED_OPP_MARGIN  | 80 cp  | Gate 3 — opponent's reply must be this much better than its alternative |
| VERIFY_DEPTH       | 18 ply | Gate 4 — depth of the confirmatory re-analysis                          |
| DETECT_TIME        | 0.10 s | Engine time per position during the detection sweep                     |
| SOLUTION_DEPTH     | 16 ply | Depth used while constructing the continuation                          |
| CONTINUATION_MOVES | 6      | Maximum half-moves appended after the player's first move               |
| MIN_PLAYER_MOVES   | 1      | Minimum player moves the puzzle must require                            |
| MAX_PER_GAME       | 5      | Hard cap on puzzles extracted from a single game                        |
| SKIP_PLIES         | 8      | Opening half-moves ignored                                              |

Gate 4 deserves particular note. Gates 1–3 operate at a fast detection time; a shallow search can be wrong. Re-analysing surviving candidates at depth 18 catches cases where the engine changes its assessment given more time — the single most important defence against emitting a puzzle whose "solution" is not actually best.

Adaptive Chess Puzzle Advisor — Technical System Report

A. Radevski · MSc AI & Data Science

### 6.3 Tactic classification and rating estimation

Accepted puzzles are labelled by a hand-written symbolic classifier that inspects the resulting position through `python-chess` [13]. It tests, in priority order, for mate, promotion, fork (two or more opponent pieces attacked, at least one heavy or loose), hanging piece, deflection, pin and skewer (a sliding piece attacking through an opponent piece toward the king), discovered attack, king safety, sacrifice (capturing with a piece worth at least 100 cp more than its target), X-ray, endgame types where twelve or fewer pieces remain, and quiet move (neither capture nor check). "General" is the terminal fallback.

The *Quiet Move* class was added specifically because approximately a third of generated puzzles were previously falling into the unlabelled "General" bucket — a category with no pedagogical meaning and no corresponding bandit arm signal.

Difficulty is estimated by a closed-form heuristic combining the severity of the error with the length of the solution:

$$rating = clamp(800 + min(\Delta eval - 150, 700) \times 0.6 + (len - 2) \times 150, 700, 2400)$$

This is uncalibrated by construction — it is computed once at mining time and never updated by how players actually perform against the puzzle. That limitation is precisely what §8 addresses.

## 7 Pipeline 3 — Adaptive Recommendation

*This is the adaptive core of the system and its most defensible claim to being genuinely AI-driven: an online Bayesian decision process that updates from every single observation.*

### 7.1 Formalisation as a Bernoulli bandit

Each of the 23 categories is modelled as an independent Bernoulli process. Each time the player attempts a puzzle in category  $i$ , the outcome is a solve (1) or a fail (0), governed by an unknown true proficiency  $p_i = P(\text{solve} \mid \text{category } i)$ . The system must estimate all 23 proficiencies online, while simultaneously using its current estimates to decide what to serve next. There is no training phase; every observation is both evidence and a pedagogical act.

### 7.2 Beta–Bernoulli conjugacy

The Beta distribution is the conjugate prior for a Bernoulli likelihood. If the belief about  $p_i$  is  $\text{Beta}(\alpha, \beta)$ , and  $s$  further successes and  $f$  further failures are observed, then the posterior is exactly:

$$p_i \mid \text{data} \sim \text{Beta}(\alpha + s, \beta + f)$$

No numerical integration, no approximation, no gradient step — two integer additions. This is what makes the update loop trivially cheap and exactly correct, and it is the entire content of the update routine:

```
if solved: arm.alpha += 1
else:      arm.beta  += 1
```

| Quantity            | Expression                                         | Interpretation                                          |
|---------------------|----------------------------------------------------|---------------------------------------------------------|
| Effective successes | $\alpha - 1$                                       | Real solves plus whatever the prior contributed         |
| Effective failures  | $\beta - 1$                                        | Real failures plus prior contribution                   |
| Total evidence      | $\alpha + \beta$                                   | Larger $\Rightarrow$ narrower, more confident posterior |
| Posterior mean      | $\alpha / (\alpha + \beta)$                        | Best point estimate of the true solve rate              |
| Posterior variance  | $\alpha\beta / [(\alpha+\beta)^2(\alpha+\beta+1)]$ | Residual uncertainty; shrinks as evidence grows         |
| Uninformative prior | $\text{Beta}(1, 1)$                                | Uniform on $[0,1]$ — the default <code>ArmState</code>  |

### 7.3 The selection rule and its deliberate inversion

Selection draws one independent sample from every arm's posterior and picks a single winner:

$$\theta_i \sim \text{Beta}(\alpha_i, \beta_i) \text{ for all } i, \text{ select } \arg \min_i \theta_i$$

### Why arg min rather than the standard arg max

Adaptive Chess Puzzle Advisor — Technical System Report

A. Radevski — MSc AI & Data Science

Canonical Thompson Sampling maximises reward: it selects the arm with the *highest* sampled value. This system inverts that, because  $\theta$  estimates *proficiency*, and the pedagogical objective is to spend scarce practice time where the player is weakest. Selecting the arg max would produce a system that serves progressively more puzzles in the categories the player has already mastered — a precise inversion of the intended behaviour. This is the one place the implementation deliberately departs from the textbook formulation<sup>[2]</sup>, and it is worth stating explicitly because it looks like an error until the objective is made clear.

Sampling rather than ranking by posterior mean is what makes exploration automatic. An arm the player has barely attempted — say Beta(1,1), mean 0.5 but maximal variance — can be sampled at 0.15 purely by chance and win selection, even though its mean looks unremarkable, precisely because the system does not yet know that mean is trustworthy. As attempts accumulate,  $\alpha + \beta$  grows, the posterior narrows, sampled values concentrate around the true mean, and exploration fades into exploitation with no schedule and no tunable  $\epsilon$ <sup>[1][20]</sup>.

### Worked example

Suppose "Fork" has been attempted 8 times and solved 3, from the default Beta(1,1). Then  $\alpha = 4$ ,  $\beta = 6$ , with posterior mean 0.4. On any given selection round the sample might return 0.28 — making Fork look weaker than its mean and likely to be chosen — or 0.55, making it look adequate so that a genuinely weaker arm wins instead. Both are legitimate draws from Beta(4,6). The randomness is not noise to be engineered away; it is the mechanism that keeps every arm in contention in proportion to how much remains unknown about it.

## 7.4 Prior seeding from the weakness profile

The weakness scores from Pipeline 1 are converted into Beta priors under a fixed pseudo-observation budget of  $\alpha + \beta = 10$ :

$$\alpha = \text{round}((1 - w) \times 10), \quad \beta = 10 - \alpha$$

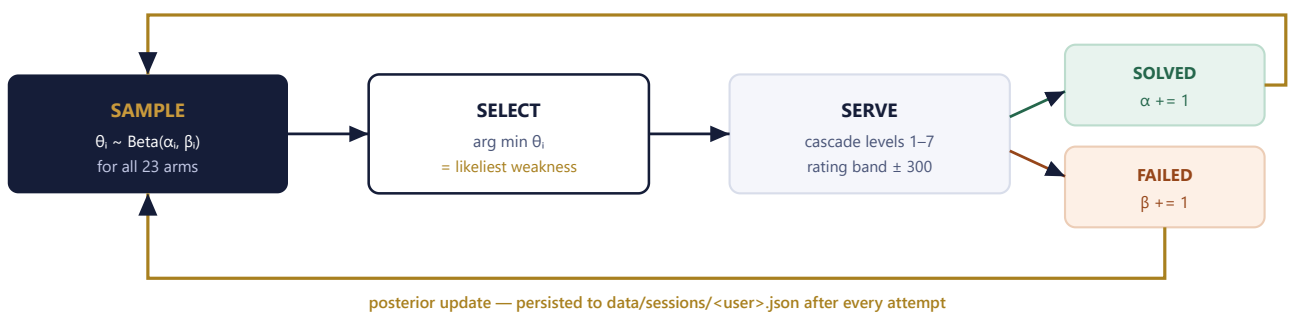
A weakness score of 0.8 becomes Beta(2, 8) — the bandit begins already believing this category is weak, but with only the confidence of ten imaginary attempts, so five or six real outcomes can already outweigh it. This is a deliberate calibration. The prior should orient the bandit without allowing a possibly-mistaken upstream profiler to dominate genuine evidence for long, which matters especially given that the profiler itself may have run in its degraded fallback mode (§5.5).

## 7.5 The serving cascade

Selecting a category is not the same as having a suitable puzzle available in it. Serving therefore proceeds through an ordered cascade, preferring personally-relevant material and degrading gracefully:

| Level | Source                              | Condition                                                                          |
|-------|-------------------------------------|------------------------------------------------------------------------------------|
| 1     | User-mined puzzles, target category | In rating band, previously unseen                                                  |
| 2     | Any unseen mined puzzle             | 40% of the time when no in-category mined puzzle exists; session target relabelled |
| 3     | Lichess pool, target category       | Requested rating window                                                            |
| 4     | Lichess pool, target category       | Rating window widened by $\pm 200$                                                 |
| 5     | Lichess pool, any category          | Requested window; target relabelled to actual                                      |
| 6     | Lichess pool, any category          | Full rating range                                                                  |
| 7     | Repeat an already-seen puzzle       | Response flagged <code>poolExhausted: true</code> rather than silently repeating   |

The rating window narrows to the player's known Elo  $\pm 300$  when available, implementing the zone-of-proximal-development constraint<sup>[18]</sup> operationally. Every result updates the bandit, appends a timestamped history record, and persists — and that history file is precisely the input the Glicko-2 fitter consumes.



**Figure 3.** The online recommendation loop. The posterior is updated and persisted after every single attempt; there is no batch training step.

## 8 Difficulty Modelling with Glicko-2

### 8.1 Why a three-parameter rating system

Puzzle difficulty in the serving path currently comes from two sources, both static: the Lichess-supplied rating for pool puzzles, and the closed-form heuristic of §6.3 for mined puzzles. Neither updates from how players in *this* system actually perform. A mined puzzle rated 1400 by formula may in practice be solved by every player who sees it, and nothing corrects that.

Glicko-2<sup>[5][6]</sup> tracks three quantities per entity rather than one:

| Symbol   | Name             | Meaning in this application                                            |
|----------|------------------|------------------------------------------------------------------------|
| $r$      | Rating           | Elo-like scale; for a puzzle, higher means harder                      |
| RD       | Rating deviation | Uncertainty; $r \pm 2 \cdot \text{RD}$ is approximately a 95% interval |
| $\sigma$ | Volatility       | How erratically true difficulty appears to move over time              |

The practical value is that a freshly mined puzzle starts with high RD and is therefore permitted to move quickly under the first few real outcomes, while an established rating resists revision — the exact behaviour a bare Elo number cannot express.

### 8.2 The update algorithm in full

All computation occurs on an internal scale obtained by centring and dividing by the Glicko-2 constant 173.7178, which keeps the logistic arithmetic numerically well-behaved:

$$\mu = (r - 1500) / 173.7178, \quad \varphi = \text{RD} / 173.7178$$

A single rating period, containing one or more results, proceeds through seven steps:

| Step | Quantity                                           | Purpose                                                                                                                                |
|------|----------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| 1    | $g(\varphi_j) = 1 / \sqrt{1 + 3\varphi_j^2/\pi^2}$ | Discount each opponent's contribution by that opponent's own uncertainty — a result against a poorly-known opponent is weaker evidence |
| 2    | $E(\mu, \mu_j, \varphi_j)$                         | Expected outcome — a logistic function of the discounted rating gap; Glicko-2's analogue of the Elo expected score                     |
| 3    | $v$                                                | Estimated variance of the rating change: how much information this batch of results actually carries                                   |
| 4    | $\Delta$                                           | Naive implied rating change — actual outcomes versus expected, scaled by $v$                                                           |
| 5    | $\sigma'$                                          | New volatility, solved numerically by the <b>Illinois</b> root-finding algorithm; the only step with no closed form                    |
| 6    | $\varphi', \mu'$                                   | Uncertainty shrinks in proportion to new evidence; the rating shifts toward observed results, weighted by updated uncertainty          |
| 7    | $(r', \text{RD}')$                                 | Convert back to the public scale                                                                                                       |

Step 5 is the implementation's principal hazard. The volatility update has no analytic solution and requires iterative root-finding on an objective balancing how surprising  $\Delta$  was against how volatile the entity has historically been. It is the step most from-scratch implementations get subtly wrong — the code carries an explicit comment distinguishing the fixed constant from the iterating bracket bounds, which are easy to conflate.

| Constant              | Value    | Role                                                             |
|-----------------------|----------|------------------------------------------------------------------|
| GLICK02_SCALE         | 173.7178 | Public ↔ internal scale conversion                               |
| DEFAULT_TAU           | 0.5      | System constant constraining volatility change (0.3–1.2 typical) |
| DEFAULT_PLAYER_RATING | 1500.0   | Seed rating for an unseen entity                                 |
| DEFAULT_PLAYER_RD     | 350.0    | Full uncertainty for a genuinely new entity                      |
| DEFAULT_VOLATILITY    | 0.06     | Glickman's own recommended default                               |

### 8.3 Adaptation to puzzle rating

The adaptation treats each recorded attempt as a rating period containing exactly one game, in which the *puzzle is the opponent*. A solve is a win for the player and a loss for the puzzle, which eases its fitted difficulty downward; a failure is a win for the puzzle, raising it. Replaying the full chronological event log through this update yields a self-consistent, evidence-driven calibration of both player ability and puzzle difficulty on the same scale.

This is structurally the same idea Lichess itself uses to rate its puzzles from aggregate solver performance<sup>[14]</sup>. The difference is the size of the evidence pool, and that difference is currently decisive — see §8.5.

### 8.4 Validation against the published reference

#### Reproduction of Glickman's canonical worked example

A player at rating 1500, RD 200, volatility 0.06 plays three opponents in one rating period: defeating a 1400/RD30 opponent, then losing to 1550/RD100 and to 1700/RD300. Glickman's published example<sup>[6]</sup> gives the post-period result as **rating**  $\approx$  **1464.06**, **RD**  $\approx$  **151.52**, **volatility**  $\approx$  **0.05999**. The implementation reproduces these values to the tested tolerance.

The result is also interpretable, which is a useful secondary check that the transcription is semantically and not merely numerically correct. The rating falls, since two losses outweigh one win against a weaker opponent. RD contracts sharply from 200 to 151.5, because three real games constitute substantial evidence. Volatility barely moves, from 0.06 to 0.05999, because none of the three outcomes was surprising given the player's prior rating — the system sees no reason to believe this competitor's underlying strength is unusually unstable.

Matching an independent third-party published reference is the strongest available evidence that the mathematics is correct, since it is entirely independent of anything else in this codebase.

## 8.5 Current yield and the honest limitation

|                                                         |  |                      |                                     |                |  |
|---------------------------------------------------------|--|----------------------|-------------------------------------|----------------|--|
| Adaptive Chess Puzzle Advisor — Technical System Report |  |                      | A. Radevski · MSc AI & Data Science |                |  |
| 105                                                     |  | 3                    |                                     | 61             |  |
| SOLVE EVENTS AVAILABLE                                  |  | PLAYERS WITH HISTORY |                                     | PUZZLES FITTED |  |

With 105 events spread across 3 players, most puzzles receive only one or two rating-period updates. That is nowhere near enough for a fitted rating to have converged to something more reliable than the static heuristic it would replace. The methodology is correct and validated; the data volume that would make it *better* than the incumbent does not yet exist. This is why the fitter remains an offline CLI producing a standalone artefact rather than a component of the serving path — a status this report states rather than implying deployment.

## 9 Evaluation

### 9.1 A two-level evaluation strategy

The system has two independently-failing parts, and conflating them obscures which one is responsible when results disappoint: the **puzzle material** (is the raw content sound?) and the **recommender** (given sound material, does it select well?). Each requires its own ground truth and its own acceptance criteria.

### 9.2 Puzzle quality: the five-dimension evaluator

A composite score is computed per puzzle across five weighted dimensions. This evaluator exists to validate the *generator*, and is deliberately separate from the gates that admitted the puzzle in the first place — it is a second, independent opinion rather than a restatement of the first.

| Dimension        | Weight | Criterion                                                            |
|------------------|--------|----------------------------------------------------------------------|
| Engine agreement | 0.30   | Stockfish at depth 18 confirms the intended move is still best       |
| Clarity          | 0.25   | PV1 – PV2 gap: full credit at $\geq 150$ cp, partial at $\geq 80$ cp |
| Solution depth   | 0.20   | At least 3 ply total                                                 |
| Non-triviality   | 0.15   | Not an undefended free capture with no follow-up                     |
| Difficulty fit   | 0.10   | $ \text{puzzle rating} - \text{player Elo}  \leq 300$                |

The CLI harness runs this over a user's mined puzzles and emits a per-metric pass-rate breakdown, a per-category analysis, best and worst examples, tuning recommendations, and a timestamped JSON report for cross-iteration comparison. Project targets are `avg_score  $\geq 0.70$`  and `engine_agrees  $\geq 75\%$` .

### 9.3 Recommender validation methodology

A player's true per-category weakness is never directly observable; only noisy solve/fail outcomes are. A controlled synthetic environment with known ground truth is therefore the only way to verify that the bandit converges to the right answer — this is the standard validation posture for bandit algorithms before deployment on live traffic<sup>[20]</sup>.

The protocol simulates a player with fixed known solve probabilities (three categories at 0.30, the remainder at 0.80), across 200 independent Monte Carlo runs of 150 attempts each, so that results carry confidence intervals rather than being single-run anecdotes. Baselines are uniform random category selection and fixed round-robin cycling — both realistic non-adaptive alternatives rather than strawmen.

Acceptance criteria, fixed in advance of running

- Adaptive Chess Puzzle Advisor — Technical Systems Report
- A. Radarski, MSc AI & Data Science
1. Weak-category targeting rate  $\geq 80\%$  by trial 150, against  $\sim 13\%$  expected by chance (3 of 23).
  2. Posterior estimation error (MAE against true solve rate)  $\leq 0.10$  within 100 trials.
  3. Statistically significant improvement over random selection — Welch's  $t$ -test,  $p < 0.01$  — with a large effect size, Cohen's  $d > 0.8$ .

Pre-registering these thresholds matters methodologically: it frames the evaluation as hypothesis-driven rather than as charts selected after the fact. A stated and reasoned failure against a pre-declared criterion is more credible than an unstated pass.

9.4 Classifier cross-validation results

Held-out-*player* five-fold cross-validation over 300 synthetic players (seed 42). Splitting by player rather than by observation is the methodologically essential choice — splitting by move would leak the same player's signal across folds and inflate results.

| Fold | Model MAE | Baseline MAE (training mean) | True top weakness in predicted top-3 |
|------|-----------|------------------------------|--------------------------------------|
| 0    | 0.1645    | 0.1674                       | 30%                                  |
| 1    | 0.1628    | 0.1649                       | 30%                                  |
| 2    | 0.1638    | 0.1674                       | 27%                                  |
| 3    | 0.1642    | 0.1677                       | 38%                                  |
| 4    | 0.1686    | 0.1710                       | 40%                                  |
| Mean | 0.1648    | 0.1677                       | 33%                                  |

The model beats the naive baseline in every fold, and its top-3 hit rate of 33% is roughly 2.5× the  $\sim 13\%$  expected by chance. The most informative features are endgame error rate (0.087), blunders per game (0.062), and the per-category Endgame error rate (0.052).

The honest reading is that this is a modest but real and consistent signal, not a dramatic one — and that is an accurate reflection of the task's difficulty. Recovering which one to three of 23 fine-grained motifs are genuinely weak, from 30 summary features, was never going to produce high accuracy. Reporting the margin as small is more useful than presenting it as more than it is.

9.5 Test suite and reference validation

128

PASSING TESTS ACROSS 8 FILES, FULLY OFFLINE

5 / 5

CV FOLDS BEATING BASELINE

The suite covers the bandit (prior seeding, update directionality, statistical arm preference under a seeded RNG, state round-tripping through persistence), the generator, the evaluator, the data loader, and the Flask routes. The Glicko-2 module is validated against Glickman's published example (§8.4).

### **What "128 passing tests" does not establish**

The suite is deliberately engine-free so it runs offline and fast. That is a real constraint on what it can catch: a defect that only manifests when a live Stockfish process is attached is invisible to it by construction. Section 11.1 documents exactly such a defect, which passed the entire suite and was caught only by a live smoke test. "The tests pass" and "it works against a real engine" are distinct claims, and this report treats them as such.

## 10 What Is Genuinely AI-Driven

### 10.1 Component-by-component breakdown

"Is this AI?" deserves a layered answer rather than a single label, because the system mixes genuine online learning, supervised learning, statistical estimation, third-party search AI, and hand-written symbolic logic. Conflating these would misrepresent the contribution in either direction.

| Component                               | Methodological category                                                       | Learned from data?                                                        |
|-----------------------------------------|-------------------------------------------------------------------------------|---------------------------------------------------------------------------|
| <b>Thompson Sampling bandit</b>         | Bayesian reinforcement learning; sequential decision-making under uncertainty | <b>Yes — online</b> Posteriors update from every real attempt             |
| <b>RandomForest weakness classifier</b> | Supervised machine learning (ensemble regression)                             | <b>Yes</b> Trained on simulated cohorts; opt-in, not default              |
| <b>Glicko-2 difficulty fitter</b>       | Bayesian statistical rating estimation                                        | <b>Yes</b> Updates from real outcomes; data volume still small            |
| Stockfish engine                        | Classical search AI — alpha-beta with NNUE evaluation <sup>[1]</sup>          | <b>Yes, but third-party</b> A dependency, not a contribution of this work |
| <code>_classify_tactic()</code>         | Symbolic reasoning over board state                                           | <b>No</b> Hand-coded geometric heuristics                                 |
| Rule-based weakness scorer              | Hand-tuned threshold rules                                                    | <b>No</b> Hand-tuned conditionals                                         |
| Heuristic fallback profiler             | Coarse win/loss heuristic                                                     | <b>No</b> Coarser still — two buckets only                                |
| Pool filtering and enrichment           | Deterministic data engineering                                                | <b>No</b>                                                                 |

### 10.2 The defensible claim

#### Where the AI genuinely is

The strongest and most defensible claim is the **Thompson Sampling bandit**. It is live in the application, it is online — updating from every single puzzle attempt with no batch training step — it maintains full posterior distributions rather than point estimates, and it is a textbook instance of Bayesian exploration/exploitation<sup>[1]</sup> applied to a real sequential recommendation problem. The RandomForest classifier is a legitimate second contribution, methodologically sound and validated, gated on data volume rather than on anything unfinished in the model itself.

It is equally important to state what this is not. It is not deep reinforcement learning: there is no neural policy network, no PPO or DQN, no learned value function. It is a *non-contextual* bandit — arms do not condition on features of the current position or session state. Naming the RL family precisely is more defensible than

the looser claim, and it forecloses the obvious objection that the architecture does not match a deep-RL description.

# 11 Limitations and Threats to Validity

## 11.1 Defects found and fixed

These were found by re-reading the live source against its own documentation. They are recorded here because a system report that omits them would misrepresent what was actually running.

### Fixed — engine detection made the primary analysis path unreachable

The analysis entry point located Stockfish with `shutil.which("stockfish")`, which searches only `PATH` for a binary of that exact name and never checked the project's own bundled binary. The rest of the codebase used a dedicated resolver that does check it. Consequently the Stockfish-verified rule-based scorer was **never reached in the live application**, and every analysis silently fell through to the two-bucket heuristic fallback. Fixed by calling the same resolver used elsewhere.

### Fixed — a second defect that the first fix exposed

Correcting engine detection meant real Stockfish calls ran against live games for the first time, which immediately surfaced a latent bug: both analysis calls passed `multipv=1`, which causes `python-chess` to return a *list* rather than a single info dictionary. The subsequent subscript then raised a `TypeError` on every analysed move, failing every game and falling through to the heuristic path again. Fixed by dropping the unnecessary argument.

This defect had zero test coverage, because the suite is deliberately engine-free and therefore never invokes a real engine call. It was caught by a live smoke test. This is the concrete evidence behind the caveat in §9.5.

### Fixed — all three scoring paths now genuinely reachable

A `WEAKNESS_MODEL` environment variable now selects between the rule-based default and the trained classifier, with automatic fallback if the model file is missing or prediction raises. This was implemented as opt-in rather than a default swap deliberately: the classifier is validated only against simulated players, so making it the default before real-player evidence exists would substitute an unproven path for a working one.

## 11.2 Open limitations

| Limitation                          | Consequence and current position                                                                                                                                                                                                                                                                          |
|-------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Data volume</b>                  | Four real players, 105 solve events. This is the binding constraint on the entire research programme: both the Glicko-2 fitter and the ML classifier are correctly built and validated but cannot be shown to outperform the incumbent rule-based scorer on real users. The gap is data, not methodology. |
| <b>No live baseline comparison</b>  | Objective 4 requires an adaptive-versus-non-adaptive comparison on real users. The evaluation harness computes everything needed except the baseline arm itself; the simulation protocol (§9.3) is specified but is not a substitute for a live A/B.                                                      |
| <b>Simulated training data</b>      | Classifier validation establishes methodological soundness, not real-world accuracy. Disclosed wherever the results are quoted.                                                                                                                                                                           |
| <b>Non-contextual bandit</b>        | Arms do not condition on position features, time of day, session fatigue, or recent history. A contextual bandit would use strictly more information; it would also require substantially more data to fit.                                                                                               |
| <b>Independence assumption</b>      | The 23 arms are modelled as independent, but tactical skills are plainly correlated — a player weak at pins is likely weak at skewers, which share geometric structure. A hierarchical or correlated prior would pool evidence across related motifs and learn faster from few observations.              |
| <b>Two difficulty-tier schemes</b>  | The Lichess pool uses seven tiers, mined puzzles five, with different boundaries and names. A puzzle rated 1150 is "Easy" from the pool but has no equivalent mined label. Cosmetic, but visible if two puzzles are compared side by side.                                                                |
| <b>Stale constant documentation</b> | The README and evaluation CLI docstring quote superseded tuning values (e.g. threshold 120 where the code uses 150). Documentation-only, but the kind of discrepancy that undermines confidence in other numbers.                                                                                         |
| <b>JSON file persistence</b>        | Adequate at current scale; offers no concurrency control, no transactional integrity, and no query capability. It would not survive meaningful multi-user load.                                                                                                                                           |
| <b>Self-selected population</b>     | Players are Chess.com users who opted in. Any performance claim generalises to that population only, not to chess learners in general.                                                                                                                                                                    |

## 12 Future Work

In priority order, reflecting the fact that the binding constraint is evidence rather than engineering:

1. **Collect real session data at volume.** Everything else is gated on this. Both offline research components become deployable the moment there are enough real solve events for their fitted outputs to beat the static incumbents.
2. **Run the live A/B of the trained classifier against the rule-based scorer.** Both paths are now genuinely reachable in the application, so this is a data-collection exercise rather than an engineering one — the comparison will be apples-to-apples now that the engine-detection defect is fixed.
3. **Wire Glicko-2 fitted ratings into the serving path,** preferring a fitted rating where its RD is below a confidence threshold and falling back to the static rating otherwise — so difficulty reflects how puzzles behave in *this* system rather than on Lichess.
4. **Execute the pre-registered bandit simulation** (§9.3) and report results against the stated criteria, including the non-adaptive baselines, closing Objective 4's automatic-evaluation arm.
5. **Model inter-category correlation.** A hierarchical prior over related motif families would let evidence about pins inform beliefs about skewers, materially improving convergence in short sessions — the regime that actually matters for a practice tool.
6. **Reconcile the two difficulty-tier schemes and refresh stale constant documentation** (§11.2) — small, and cheap relative to the credibility cost of leaving them.

## 13 Conclusion

This report has documented a complete, working adaptive chess training system: from public game-history ingestion, through engine-based error analysis and weakness profiling, through engine-verified puzzle mining, to an online Bayesian recommendation loop that updates its beliefs after every attempt and persists them across sessions.

The system's central technical claim is narrow and defensible. A Thompson Sampling bandit over 23 tactical categories, with priors seeded from Stockfish-derived error statistics and an inverted selection rule that targets weakness rather than reward, is a genuine application of Bayesian sequential decision-making<sup>[1][2]</sup> to a real pedagogical problem. It is live, it is online, and its correctness is unit-tested. Around it sit two further components — a from-scratch Glicko-2 fitter validated against Glickman's published reference<sup>[6]</sup>, and a RandomForest weakness classifier validated by held-out-player cross-validation — both methodologically sound and both explicitly gated on real-player data volume rather than presented as deployed.

The most significant finding of the review that produced this report was not an algorithmic result but a methodological one: two defects had made the system's primary analysis path unreachable, so the sophisticated scorer being documented was not the one actually running. Recording that, along with the fact that a 128-test offline suite could not have caught it, is more useful to the work than a cleaner narrative would have been. The gap between what a system is designed to do and what it demonstrably does is exactly the gap an academic system report exists to measure.

## 14 References

Each entry notes where in the system it is applied. All bibliographic details were verified against the publisher or primary source of record.

- [1] Thompson, W. R. (1933). On the likelihood that one unknown probability exceeds another in view of the evidence of two samples. *Biometrika*, 25(3–4), 285–294. doi:10.1093/biomet/25.3-4.285  
*Applied in: §7 — the originating formulation of the posterior-sampling rule used by the recommendation core.*
- [2] Russo, D. J., Van Roy, B., Kazerouni, A., Osband, I., & Wen, Z. (2018). A tutorial on Thompson sampling. *Foundations and Trends in Machine Learning*, 11(1), 1–96. doi:10.1561/22000000070  
*Applied in: §2.2, §7.3 — modern regret analysis and the standard formulation from which this system's arg-min inversion departs.*
- [3] Auer, P., Cesa-Bianchi, N., & Fischer, P. (2002). Finite-time analysis of the multiarmed bandit problem. *Machine Learning*, 47(2–3), 235–256. doi:10.1023/A:1013689704352  
*Applied in: §2.2 — the UCB alternative against which Thompson Sampling was selected.*
- [4] Sutton, R. S., & Barto, A. G. (2018). *Reinforcement Learning: An Introduction* (2nd ed.). Cambridge, MA: MIT Press. ISBN 978-0-262-03924-6.  
*Applied in: §2.2, §10.2 — standard reference for the bandit formalism and for locating this work within the RL family.*
- [5] Glickman, M. E. (1999). Parameter estimation in large dynamic paired comparison experiments. *Journal of the Royal Statistical Society: Series C (Applied Statistics)*, 48(3), 377–394. doi:10.1111/1467-9876.00159  
*Applied in: §8 — the rating-deviation concept underpinning uncertainty-aware difficulty estimation.*
- [6] Glickman, M. E. (2012). *Example of the Glicko-2 system* [Technical note]. Boston University. [glicko.net/glicko/glicko2.pdf](http://glicko.net/glicko/glicko2.pdf)  
*Applied in: §8.2, §8.4 — the algorithm specification implemented from scratch, and the worked example used as the validation reference.*
- [7] Elo, A. E. (1978). *The Rating of Chessplayers, Past and Present*. New York: Arco Publishing.  
*Applied in: §2.1 — the baseline rating formulation whose single-parameter limitation motivates the Glicko-2 work.*
- [8] Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32. doi:10.1023/A:1010933404324  
*Applied in: §5.4 — the ensemble method used by the trained weakness classifier.*
- [9] Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.  
*Applied in: §5.4, §9.4 — the implementation of the classifier and its cross-validation protocol.*
- [10] Silver, D., Hubert, T., Schrittwieser, J., Antonoglou, I., Lai, M., Guez, A., ... Hassabis, D. (2018). A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play. *Science*, 362(6419), 1140–1144. doi:10.1126/science.aar6404  
*Applied in: §2.3, §10.2 — the contrasting deep-RL approach this project explicitly does not attempt, used to delimit the claim made here.*
- [11] Nasu, Y. (2018). *ЗУИИ: Efficiently updatable neural-network-based evaluation functions for computer shogi*. Ziosoft Computer Shogi Club.  
*Applied in: §2.3 — the NNUE evaluation architecture later adopted by Stockfish, which supplies all engine ground truth in this system.*

- [12] Romstad, T., Costalba, M., Kiiski, J., Linscott, G., et al. *Stockfish: A free and strong UCI chess engine* [Computer software]. [stockfishchess.org](https://stockfishchess.org)

Adaptive Chess Puzzle Advisor — Technical System Report A. Badawski — MSc AI & Data Science  
Applied in: §5.2, §6, §9.2 — the analysis oracle for centipawn loss, puzzle verification gates, and independent quality scoring.

- [13] Fiekas, N. *python-chess: A chess library for Python* [Computer software]. [python-chess.readthedocs.io](https://python-chess.readthedocs.io)

Applied in: §4.5, §5.1, §6.3 — board representation, legality, PGN parsing, and UCI engine communication.

- [14] Lichess. *lichess.org open database — puzzles* [Dataset, CC0 1.0]. [database.lichess.org/#puzzles](https://database.lichess.org/#puzzles)

Applied in: §4.1–4.3 — the ~6M-puzzle base corpus, its Glicko-2 ratings, and its theme tags.

- [15] Chess.com. *Published-Data API documentation*. [chess.com/news/view/published-data-api](https://chess.com/news/view/published-data-api)

Applied in: §4.5 — player profile and game archive ingestion, including the rate-limit guidance followed by the client.

- [16] McKinney, W. (2010). Data structures for statistical computing in Python. In *Proceedings of the 9th Python in Science Conference* (pp. 56–61). doi:10.25080/Majora-92bf1922-00a

Applied in: §4.2 — the chunked ETL and Parquet row-group streaming that bound memory over the 1.1 GB corpus.

- [17] Corbett, A. T., & Anderson, J. R. (1995). Knowledge tracing: Modeling the acquisition of procedural knowledge. *User Modeling and User-Adapted Interaction*, 4(4), 253–278. doi:10.1007/BF01099821

Applied in: §1.2, §2.4 — the per-skill Bayesian mastery model this system's arm posteriors structurally parallel.

- [18] Vygotsky, L. S. (1978). *Mind in Society: The Development of Higher Psychological Processes*. Cambridge, MA: Harvard University Press.

Applied in: §1.2, §7.5 — the zone of proximal development, implemented operationally as the player-Elo  $\pm 300$  serving band.

- [19] Ericsson, K. A., Krampe, R. T., & Tesch-Römer, C. (1993). The role of deliberate practice in the acquisition of expert performance. *Psychological Review*, 100(3), 363–406. doi:10.1037/0033-295X.100.3.363

Applied in: §1.2 — the pedagogical premise that practice must target current deficiency to produce expertise.

- [20] Chapelle, O., & Li, L. (2011). An empirical evaluation of Thompson sampling. In *Advances in Neural Information Processing Systems 24* (pp. 2249–2257).

Applied in: §2.2, §9.3 — empirical justification for choosing Thompson Sampling over UCB, and the simulation-based validation posture.

## 15 Appendices

### 15.1 Appendix A — Literature-to-component mapping

A defence-oriented cross-reference: for each implemented component, the primary source that justifies the design choice.

| Implemented component                      | Source file                                                                          | Grounding reference                                                                                         |
|--------------------------------------------|--------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| Thompson Sampling bandit                   | <code>recommender/bandit.py</code>                                                   | Thompson 1933 <sup>[1]</sup> ; Russo <i>et al.</i> 2018 <sup>[2]</sup> ; Chapelle & Li 2011 <sup>[20]</sup> |
| Beta-Bernoulli conjugate update            | <code>recommender/bandit.py</code>                                                   | Thompson 1933 <sup>[1]</sup> ; Sutton & Barto 2018 <sup>[4]</sup>                                           |
| Choice of TS over UCB / $\epsilon$ -greedy | Design decision, §2.2                                                                | Auer <i>et al.</i> 2002 <sup>[3]</sup> ; Chapelle & Li 2011 <sup>[20]</sup>                                 |
| Per-skill mastery posture                  | <code>classifier/player_profiler.py</code>                                           | Corbett & Anderson 1995 <sup>[17]</sup>                                                                     |
| Weakness-targeted practice rationale       | Whole-system objective                                                               | Ericsson <i>et al.</i> 1993 <sup>[19]</sup>                                                                 |
| Elo $\pm$ 300 serving band                 | <code>data/puzzle_difficulty_mapper.py</code>                                        | Vygotsky 1978 <sup>[18]</sup>                                                                               |
| Glicko-2 implementation                    | <code>analysis/difficulty_fitter.py</code>                                           | Glickman 1999 <sup>[5]</sup> ; Glickman 2012 <sup>[6]</sup>                                                 |
| Glicko-2 validation fixture                | <code>tests/test_difficulty_fitter.py</code>                                         | Glickman 2012 worked example <sup>[6]</sup>                                                                 |
| Rating-system baseline critique            | §2.1, §8.1                                                                           | Elo 1978 <sup>[7]</sup>                                                                                     |
| RandomForest classifier                    | <code>classifier/ml_weakness_model.py</code>                                         | Breiman 2001 <sup>[8]</sup> ; Pedregosa <i>et al.</i> 2011 <sup>[9]</sup>                                   |
| Engine analysis / verification             | <code>classifier/stockfish_analyzer.py</code> ,<br><code>puzzles/generator.py</code> | Stockfish <sup>[12]</sup> ; Nasu 2018 <sup>[11]</sup>                                                       |
| Board logic, PGN, UCI                      | Throughout                                                                           | Fiekas, python-chess <sup>[13]</sup>                                                                        |
| Puzzle corpus                              | <code>data/puzzle_loader.py</code>                                                   | Lichess Open Database <sup>[14]</sup>                                                                       |
| Game-history ingestion                     | <code>api/chess_com_fetcher.py</code>                                                | Chess.com Published-Data API <sup>[15]</sup>                                                                |
| Chunked ETL                                | <code>scripts/build_puzzle_dataset.py</code>                                         | McKinney 2010 <sup>[16]</sup>                                                                               |
| Scope boundary vs deep RL                  | §10.2                                                                                | Silver <i>et al.</i> 2018 <sup>[10]</sup>                                                                   |

## 15.2 Appendix B — Complete constants reference

| Engine analysis |        |
|-----------------|--------|
| ANALYSIS_TIME   | 0.05 s |
| INACCURACY_CP   | 50     |
| MISTAKE_CP      | 100    |
| BLUNDER_CP      | 200    |

| Puzzle mining      |        |
|--------------------|--------|
| PUZZLE_THRESHOLD   | 150 cp |
| MIN_CLARITY_CP     | 100 cp |
| FORCED_OPP_MARGIN  | 80 cp  |
| DETECT_TIME        | 0.10 s |
| VERIFY_DEPTH       | 18 ply |
| SOLUTION_DEPTH     | 16 ply |
| CONTINUATION_MOVES | 6      |
| MIN_PLAYER_MOVES   | 1      |
| MAX_PER_GAME       | 5      |
| SKIP_PLIES         | 8      |
| MATE_CP            | 9,000  |

| Glicko-2              |          |
|-----------------------|----------|
| GLICKO2_SCALE         | 173.7178 |
| DEFAULT_TAU           | 0.5      |
| DEFAULT_PLAYER_RATING | 1500.0   |
| DEFAULT_PLAYER_RD     | 350.0    |
| DEFAULT_VOLATILITY    | 0.06     |

| Bandit & serving              |                  |
|-------------------------------|------------------|
| Arms                          | 23               |
| Default arm state             | Beta(1, 1)       |
| Prior budget $\alpha + \beta$ | 10               |
| Rating band                   | Elo $\pm$ 300    |
| Band widening                 | $\pm$ 200        |
| Mined-puzzle blend            | 40%              |
| Opening phase                 | $\geq$ 28 pieces |
| Middlegame phase              | 14–27 pieces     |
| Endgame phase                 | $<$ 14 pieces    |

## 15.3 Appendix C — Repository map

```
src/
├─ api/chess_com_fetcher.py      Chess.com client + disk cache
├─ data/
│   ├─ pgn_parser.py            PGN → structured dict; game-phase classification
│   ├─ puzzle_loader.py         Lichess load/enrich/clean; WEAKNESS_CATEGORIES
│   ├─ puzzle_difficulty_mapper.py Rating-band targeting (ZPD)
│   └─ data_cleaner.py          Dedup, calibration and quality filters
├─ classifier/
│   ├─ stockfish_analyzer.py    Per-move analysis → MoveError / GameAnalysis
│   ├─ player_profiler.py       build_profile() – rule-based scorer (DEFAULT)
│   └─ ml_weakness_model.py     build_profile_ml() – RandomForest (opt-in)
├─ analysis/
│   ├─ style_profile.py         Engine-free archetype classification
│   └─ difficulty_fitter.py     Glicko-2 fitting (offline CLI)
├─ recommender/bandit.py        ThompsonBandit – the live adaptive core
└─ puzzles/generator.py         Four-gate Stockfish puzzle miner

eval/
├─ puzzle_evaluator.py          Five-dimension composite quality score
├─ run_evaluation.py            CLI harness → timestamped JSON reports
└─ reports/                     Saved evaluation runs

web/
├─ backend/app.py               Flask API; serves the frontend same-origin
└─ frontend/index.html          Vanilla JS SPA

scripts/build_puzzle_dataset.py  One-shot streaming ETL
tests/                           128 tests across 8 files, fully offline
```

## 15.4 Appendix D — Reproduction guide

```
# Environment
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt

# Build the puzzle pool (requires DataSets/lichess_db_puzzle.csv)
python scripts/build_puzzle_dataset.py

# Run the application – Flask serves API and frontend from one origin
python web/backend/app.py          # → http://localhost:5000

# Opt into the trained weakness classifier (PowerShell)
$env:WEAKNESS_MODEL="ml"; python web/backend/app.py

# Test suite – 128 tests, no engine or network required
pytest tests/ -v

# Offline research CLIs
python -m src.analysis.difficulty_fitter      # Glicko-2 fit from data/sessions/*.json
python -m src.classifier.ml_weakness_model    # train + cross-validate the classifier

# Puzzle quality report for a user with mined puzzles
python eval/run_evaluation.py --username <name> --elo <elo>
```

### Verification note

Every constant, metric, category count and behavioural claim in this report was read directly from the source tree at the time of writing, and every bibliographic entry in §14 was verified against the publisher or primary source of record. Where the code and the project's earlier documentation disagree, the code is treated as authoritative and the discrepancy is recorded in §11.2.